

HBalloc: A High-Performance Persistent Memory Allocator via Nonblocking Global Allocation and Batched Persistence

Yujie Hu¹, Graduate Student Member, IEEE, Yu Hua², Senior Member, IEEE, Xiangyu Xiang, Aoyang Tong³, and Xue Liu⁴, Fellow, IEEE

Abstract—The persistent memory (PM) allocators are essential components in PM-oriented programs, which are responsible for efficiently managing the persistent heap and dynamically allocating or reclaiming PM objects. However, the existing PM allocators suffer from severe performance degradation when the size of allocated object increases, primarily due to high lock overheads. They also have to repeatedly flush the same cacheline when consecutively allocating small objects from the same memory chunk, thus causing high persistence costs. Moreover, the offset-based persistent pointers that have been widely used in the existing PM allocators are location-dependent, which cannot support object migration and therefore cannot support defragmentation. In order to address these challenges, we propose HBalloc, an efficient PM allocator that leverages nonblocking global allocations to eliminate high lock overheads and employs batched persistence to avoid repeated cacheline flushes. Moreover, we propose position-independent persistent pointers (PIPs) to support object migration and lay the foundation for defragmentation. HBalloc is implemented as a user-level library that is independent of specific PM platforms. We evaluate the performance of HBalloc on a real PM server. Extensive evaluation results on multiple microbenchmarks and real-world applications demonstrate that HBalloc significantly improves the performance of both small and large object allocations, compared to the state-of-the-art PM allocators.

Index Terms—Allocator, batched persistence, crash consistency, nonblocking allocation, persistent memory (PM).

I. INTRODUCTION

BYTE-ADDRESSABLE persistent memory (PM) exhibits the salient features of both low latency and high capacity [5], [26], [40], [47]. Moreover, the PM can be memory-mapped [9], [37] to the userspace as a persistent heap [10] via direct access (DAX) mechanism [32], [42]. Applications further obtain memory space from the persistent heap and directly access PM via load and store instructions, without the

intervention of operating systems [12], [42]. The PM allocators are designed to efficiently manage the persistent heap and dynamically allocate or reclaim memory space for applications. Specifically, when a user calls the memory allocation function, the PM allocator locates a contiguous free memory space of the required size, called an *object*, in the persistent heap, and returns the persistent pointer of the object. Similarly, when the user invokes the memory reclamation function, the PM allocator reclaims the object. Hence, the PM allocators are the essential components in PM-oriented programs [39], [56], [57] that need to exploit the persistent heap. What is more, the PM allocators have to guarantee the crash consistency of the persistent heap metadata and persistently reference allocated objects across runs due to the persistence property of PM [53]. Despite the improvements over years, the performance of PM allocators has hit a bottleneck [4], [15], [44], [49], primarily due to the following three challenges.

- 1) *Size Sensitivity*: The existing PM allocators exhibit unstable performance influenced by allocation sizes. They tend to deliver much higher throughputs when the allocation size is small. However, when the allocation size increases, the throughputs would dramatically decrease by orders of magnitude, due to the high lock overheads in global allocations.
- 2) *High Persistence Tax*: The existing PM allocators suffer from high persistence tax on ensuring the crash consistency, since PM allocators frequently perform write-ahead log (WAL) and cache-line-write-back (CLWB) operations. Moreover, the same metadata entry would be repeatedly logged and persisted when consecutive small allocations obtain neighboring objects from the same memory chunk.
- 3) *Location-Dependent Persistent Pointers*: The existing PM allocators typically employ offset-based persistent pointers, which are location-dependent and cannot support object migration, since the offset of an object would change when the object moves from one location to another. This limits the performance of defragmentation in PM allocators.

To address these challenges, we analyze the essential reasons and propose HBalloc, an efficient PM allocator that solves these problems simultaneously. HBalloc employs a nonblocking global allocation mechanism, which distributes multiple working threads to different memory regions and executes global allocations in parallel, significantly reducing the overhead of concurrence control in global allocations.

Received 22 August 2026; accepted 22 August 2026. This work was supported in part by the National Natural Science Foundation of China (NSFC) under Grant 62125202 and Grant U22B2022 and in part by Hubei Provincial Natural Science Foundation of China under Grant 2026AFA002. This article was presented at the International Conference on Compilers, Architectures, and Synthesis for Embedded Systems (CASES) 2026 and appeared as part of the ESWEK-TCAD Special Issue. This article was recommended by Associate Editor S. Dailey. (*Corresponding author: Yu Hua.*)

Yujie Hu, Yu Hua, Xiangyu Xiang, and Aoyang Tong are with the School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: csyhua@hust.edu.cn).

Xue Liu is with the School of Computer Science, McGill University, Montreal, QC H3A 0E9, Canada.

Digital Object Identifier 10.1109/TCAD.2026.3727519

HBalloc batches multiple consecutive allocations from the same memory chunk and persists the chunk metadata only once by generating a volatile copy of the chunk metadata in DRAM. The batched persistence design significantly reduces the costs of WALs and CLWBs and effectively avoids repeated cacheline flushes, while also ensuring the crash consistency. Moreover, we propose position-independent persistent pointers (PIPs) to support object migration and lay the foundation for memory defragmentation.

HBalloc is implemented as a user-level library that is independent of specific PM platforms since we never use any third-party library. HBalloc memory-maps the persistent heap without kernel page cache via DAX mechanism [27], [32], [42]. Hence, it can work on top of DAX-enabled file systems (e.g., EXT4-DAX). We evaluate HBalloc on a real PM server equipped with Intel Optane DC PMMs [13]. Extensive evaluation results demonstrate that HBalloc improves the throughputs of small and large allocations by up to 20× and 160×, respectively, compared with the state-of-the-art PM allocators. We also conducted experiments on a DRAM-SSD platform to examine other forms of nonvolatile memory (NVM). Moreover, we seamlessly integrated HBalloc into the FPTree [36], a real-world application. HBalloc significantly improves the performance of fourth-order and 1024th-order FPTree by up to 20% and 50%, respectively.

In summary, this article makes the following contributions.

- 1) To address the *size sensitivity* challenge, we present the size-sensitive problem of existing PM allocators and propose a nonblocking PM allocator called HBalloc, which is not so sensitive to the allocation size.
- 2) To address the *high persistence tax* caused by frequent WAL logging and cache-line write-backs, we propose batched persistence mechanism to eliminate repeated cacheline flushes and reduce the persistence tax, while also ensuring the crash consistency of persistent heap.
- 3) To address the *location-dependent persistent pointers* challenge, we propose position-independent persistent pointers to support object migrations in the persistent heap and therefore lay the foundation for memory defragmentation.
- 4) We evaluate HBalloc on a real PM platform to show significant performance improvements upon the state-of-the-art PM allocators in both benchmarks and real applications. We release the open-source codes of HBalloc for public use <https://github.com/HUSTyjh/HBalloc>

II. BACKGROUND

A. PM and DAX

Byte-addressable PM coalesces the low latency of DRAM and the large capacity of hard disk, while also retaining data after power-offs. The PM is widely used for building high-performance file systems [24], [50], index structures [8], [60], transactional memory [22], [38], and in-memory database systems [14], [34]. Even though the commercial PM product, Intel Optane DC PMM [13], [52], has been discontinued [11], academic researches on PM are still ongoing [6], [23], i.e., the emerging CXL-based memory that can approximate PM by providing DRAM with flash-based durability [25], [41], [58]. HBalloc is designed on top of widely adopted system abstractions in PM, including byte-addressable load/store

access, persistence ordering primitives, such as CLWB and SFENCE [42], and memory-mapped persistent regions enabled by DAX [27], [32], [42]. These abstractions are supported across NVDIMM-based systems [42] and emerging CXL-attached memory [41], [58], without requiring device-specific mechanisms.

There are two approaches to access PM. Applications can utilize POSIX I/O operations [e.g., `open()`/`close()` and `read()`/`write()`] to access PM, such as accessing traditional hard disks. This approach conceals the underlying details of PM and ensures compatibility with legacy programs, simply treating the PM as a faster disk. On the other hand, with the advent of DAX mechanism [27], [32], [42], applications can exploit PM more radically by memory-mapping [9], [37] the PM to the userspace as a persistent heap [10], [31] without kernel page cache [12], [42]. Applications can further directly access the persistent heap via load and store instructions in the user mode, eliminating the overhead of trapping into the OS kernel and going through the I/O stack. DAX mechanism improves PM read and write performance by up to 56% and 81%, respectively [49].

B. Persistent Heap Management

Both DRAM and PM heap allocators typically employ some widely used techniques and terminologies, such as *size class*, *chunk*, *block*, and *tcache*. In HBalloc, *size class* determines allocation categories, *block* is the allocation unit visible to applications, *tcache* provides thread-local fast allocation, and *chunk* is the intermediate resource containers connecting thread-local allocation and the global heap. The PM allocators further have to include *crash consistency* and *persistent pointer*.

Size Class: To normalize the allocation and simplify the heap management, heap allocators usually predefine a set of standard sizes (e.g., {64, 128, 192, 256, ..., 8192} bytes), known as size classes [30], [48]. If the size of an allocation request does not match any size class, the allocator aligns it up to the nearest size class. Small allocations refer to the allocation requests not exceeding the maximum size class, while large allocations refer to ones surpassing the maximum size class. The specific size boundaries vary among allocators, depending on their internal size class definitions.

Chunks and Blocks: Modern allocators follow the two-level memory management strategy, i.e., the heap is logically divided into some equal-sized chunks (or superblocs). A chunk can be further divided into some smaller equal-sized blocks according to its size class. Chunks are the basic units of global allocations, whereas blocks are the objects finally allocated and returned to the user. In terms of allocation requests that are larger than the size of a chunk, the allocator allocates and directly returns some contiguous chunks.

Tcache: Modern allocators typically set up a thread-local memory cache (i.e., *tcache*) for each working thread. Each *tcache* contains some free chunks or blocks that are allocated from the global heap and private to the owner thread of this *tcache*, which means that other threads cannot allocate memory from this *tcache*. Each thread has its own *tcache* and allocates memory from its own *tcache* without being interrupted by other threads. Thus, a lock is not necessary

when performing local allocations from the tcache. When the tcache is exhausted, the thread allocates some free memory space from the global heap to replenish it. Since most of the small allocation requests can be served with the tcache, the concurrence control overhead is significantly reduced. However, large allocations bypass the tcache and directly allocate contiguous free chunks from the heap.

Crash Consistency: Compared to DRAM allocators, PM allocators have to address an additional challenge, i.e., guaranteeing the crash consistency of metadata. Some metadata entries that need to be updated atomically may be partially flushed back to PM from the volatile CPU cache before a power failure, and the remaining part that has not been flushed back to PM will be lost. This leads to some problems, such as runtime error, memory leak, and deadlock. What is worse, these problems cannot be addressed by restarting the system due to the persistence property of PM. In other words, the corruption of metadata is irreversible. To guarantee the crash consistency of metadata, PM allocators typically employ write-ahead-logs (WALs) and explicitly flush cachelines back to PM via CLWB instructions. Besides, memory barrier instructions (e.g., SFENCE) are used to ensure the log is persisted before the metadata.

Persistent Pointer: Due to the persistence of PM, the allocated objects in the persistent heap need to be referenced across runs. However, the persistent heap may be mapped to different virtual addresses in different runs. Hence, the virtual addresses of allocated objects will also change. To reference PM objects across runs, the existing PM allocators typically employ offset-based persistent pointers that hold the offsets of objects relative to the start of the persistent heap, based on the insight that the offset of an object relative to the start of the persistent heap keeps constant even if the persistent heap is remapped to different addresses, as long as the object is not moved.

III. MOTIVATION

In this article, we conduct experiments on a real PM platform to demonstrate the aforementioned challenges, including the size sensitivity problem, the high persistence tax, and the location-dependent persistent pointer. We evaluate multiple state-of-the-art PM allocators, including PMDK [12], [42], nvm_malloc [44], NValloc-log [15], and PMA [49], by using some widely used benchmarks, such as threadtest [2], shbench [33], larsen [28], and prod-con [43].

A. Size Sensitivity

We use the existing PM allocators [12], [15], [42], [44], [49] to allocate and reclaim 10^6 objects of the same size with 128 working threads, repeating the process for 10^3 iterations. Fig. 1 illustrates the throughputs for various allocation sizes. When the allocation sizes increase from 1 B to 64 kB, the throughputs of PMDK, nvm_malloc, NValloc-log, and PMA significantly decrease by factors of $1626\times$, $7073\times$, $48\times$, and $14524\times$, respectively. More interestingly, the performance degradations are not linear but exhibit sharp declines at certain specific sizes. For instance, the throughput of nvm_malloc decreases by a factor of 200 when the allocation size increases from 1 to 2 kB, and the throughput of PMA decreases by a factor of 247 when the allocation size increases from

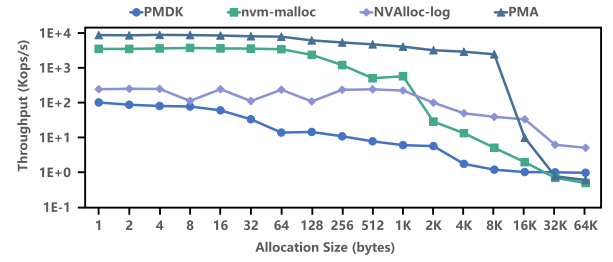


Fig. 1. Throughputs of the existing PM allocators for various allocation sizes.

8 KB to 16 KB. We further utilize the threadtest [2], shbench [33], larsen [28], and prod-con [43] benchmarks to evaluate these allocators. For each benchmark, we conduct tests for small sizes (8–256 B), medium sizes (256 B–8 kB), and large sizes (>8 kB). Fig. 2 shows the throughputs of the existing allocators across various benchmarks and size ranges. It is evident that the throughputs for large sizes are significantly lower than those for small sizes across all allocators and benchmarks. The performance gaps can be as substantial as $1992\times$, $142\times$, $2752\times$, and $15303\times$ for PMDK, nvm_malloc, NValloc-log, and PMA, respectively.

B. High-Persistence Tax

The existing PM allocators typically utilize WAL to guarantee the crash consistency of metadata. In addition, the PM allocators employ CLWB and memory-fence instructions [42] to ensure that they flush the log entry to PM before the corresponding metadata entry. However, the use of WALs and CLWB instructions introduces additional PM writes, leading to suboptimal runtime performance. This problem becomes especially exacerbated in the case of small allocations.

We evaluate the existing PM allocators using multiple benchmarks with various allocation sizes, and the results are depicted in Fig. 3. The experimental results demonstrate that the WAL overheads account for up to 84%, 61%, and 63% for nvm_malloc, NValloc-log, and PMA, respectively. The significant overheads primarily stem from repeated cacheline flushes. Due to the prevalent use of the two-level memory management strategy (Section II-B), consecutive small allocations with the same size typically obtain blocks from the same memory chunk. Consequently, the allocator repeatedly logs and modifies the chunk header, which contains the metadata (e.g., a bitmap) of the chunk. Each time a small block is allocated from the chunk, the allocator repeatedly flushes the same chunk header and the corresponding log entry from the volatile CPU cache back to PM. According to previous conclusions [7], [15], the latency of a cacheline reflush is $7.5\times$ higher than that of a normal cacheline flush. As a result, the repeated logging and cacheline reflushes introduce substantial overheads in consecutive small allocations.

C. Location-Dependent Persistent Pointer

Due to the address space layout randomization (ASLR) mechanism [45], the system may map the persistent heap to different virtual addresses across runs, resulting in changes to the addresses of allocated objects. Consequently, the applications cannot use traditional volatile pointers holding virtual addresses to reference persistent objects. To address this problem, the existing PM allocators employ offset-based persistent

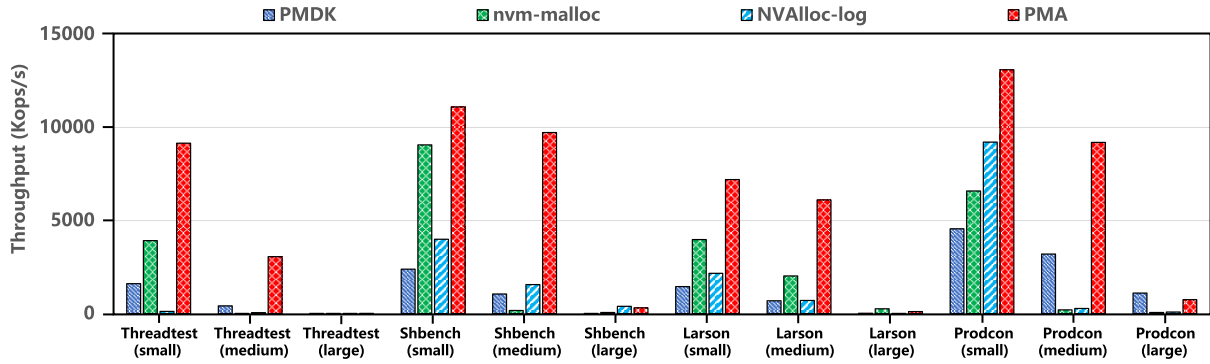


Fig. 2. Throughputs of the existing PM allocators for various benchmarks.

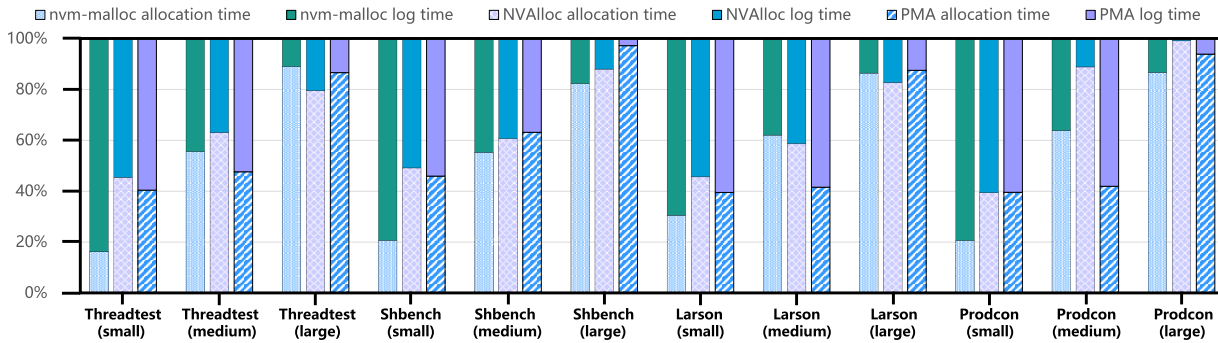


Fig. 3. Log overhead ratios of the existing PM allocators for various benchmarks.

pointers [44], [49] to reference allocated objects across runs, based on the insight that the offset of an object relative to the start of the persistent heap keeps constant even if the persistent heap is remapped. A persistent pointer holds the offset of an object and can be easily dereferenced by adding current base address of the persistent heap. However, the use of offset-based persistent pointers implies that all allocated objects in the persistent heap can never be moved, since the movement of an object leads to the change of its offset. This restriction limits the practical use of PM. Hence, it is important to propose a new type of persistent pointers to support object migrations.

IV. DESIGN OF HBALLOC

A. Overview

Fig. 4 depicts the high-level architecture of HBalloc. Different from the existing schemes, HBalloc does not directly manage the persistent heap as a whole. In our design, HBalloc logically partitions the persistent heap into several independent regions, each of which is self-contained. HBalloc manages each region separately. This approach not only simplifies the global memory management but also lays the groundwork for nonblocking global allocation. HBalloc predefines some standard size classes ranging from 64 B to 8 kB, determined by a suitable series, which keeps the internal fragmentation ratio below 20% [30]. Allocation requests under 8 kB fall into the category of small allocations and benefit from the tcache, while allocation requests larger than 8 kB are considered as large allocations and trigger global allocations from the global persistent heap, bypassing the tcache. HBalloc triggers a global allocation when a tcache becomes exhausted. To handle the scenario where multiple working threads simultaneously perform global allocations, HBalloc utilizes hashing to distribute

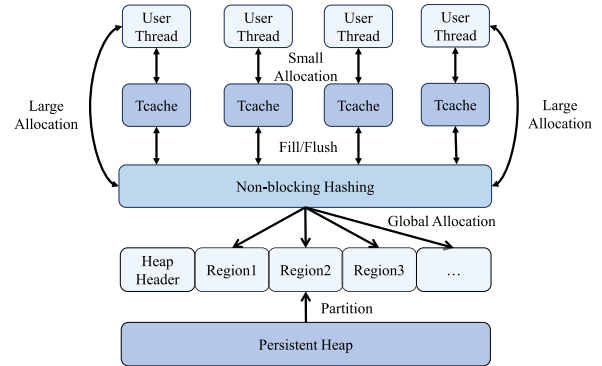


Fig. 4. Overview of HBalloc.

working threads to different regions of the persistent heap, facilitating parallel execution of global allocations.

B. Nonblocking Global Allocation

HBalloc utilizes tcache to mitigate contentions among multiple working threads. Small allocations not surpassing the predefined size classes are served by the tcache. The working thread can directly acquire a free memory block from its tcache without being blocked by other threads, since the tcache is private to this thread and inaccessible by others. However, when the tcache is exhausted or the allocation size exceeds the maximum size class, the global allocation will be triggered to allocate some free chunks from the global persistent heap. HBalloc uses these chunks either to replenish an exhausted tcache or directly fulfill the large allocation request.

Global allocations incur the potential of contentions among multiple threads, which is typically addressed by employing

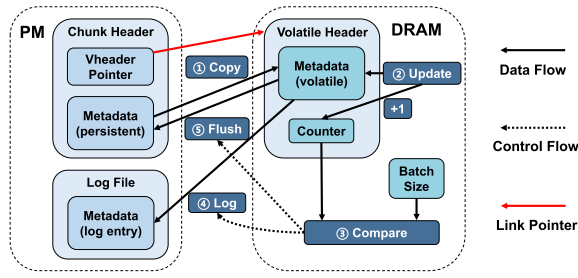


Fig. 5. Key data structure and workflow of the batched persistence mechanism.

mutex locks. However, the use of mutex locks leads to the blocking of working threads and frequent switches between the user and kernel modes, significantly impacting the performance of applications. Consequently, we choose to abandon the mutex locks and propose a nonblocking global allocation scheme. As shown in Fig. 4, global allocations will bypass the tcache layer but go through a hashing layer. The hashing layer distributes multiple working threads to different regions of the global persistent heap according to their thread IDs, allowing to perform the global allocations in parallel. Hence, HBalloc partitions the whole persistent heap into multiple independent regions. It is worth noting that the number of regions needs to be carefully determined. Specifically, if the number of regions is too small, the probability of hash collisions would increase dramatically. On the other hand, if the number of regions is too large, the persistent heap will be fragmented. Thus, we have to make a tradeoff between the hash collision and the memory fragmentation. We will evaluate the impact of the region size on the performance of HBalloc in Section VI-B3.

To entirely avoid mutex locks, we utilize the compare-and-swap (CAS) instruction, which is an atomic instruction to achieve nonblocking concurrency control in the user mode [30]. In our design, we assign a flag to each region and utilize the CAS instruction to check and modify the flag atomically. When a working thread invokes the global allocation procedure, it calculates a hash value based on its TID. The hash value indicates the index of the region that this thread is assigned to. However, due to the hash collisions, the hashing layer may distribute multiple threads to the same region. Hence, the thread checks whether another thread already occupies the region via the CAS instruction. If the CAS instruction returns false, it means that this region has been already occupied by other thread. The thread then turns to check other regions by recalculating the hash value. When the CAS instruction returns true, it allocates a free chunk from this region and frees this region by resetting the flag after the allocation.

C. Batched Persistence

To mitigate the overhead of frequent write-ahead-logs (WALs) and eliminate repeated cacheline flushes that introduce high latency on the critical path, we propose the batched persistence mechanism in HBalloc. As shown in Fig. 5, HBalloc assigns a volatile header (vheader) in DRAM to each chunk header, which stores the core metadata of a chunk. HBalloc creates a vheader and references it through a pointer in the chunk header, once the chunk is allocated from the persistent heap. The vheader contains a copy of metadata in the

corresponding chunk header (①) and an update counter. Each time a block is allocated/reclaimed from/to a chunk, HBalloc does not directly update the metadata in the corresponding chunk header in PM. Instead, HBalloc updates the copy of the metadata in the corresponding vheader in DRAM and increases the update counter by one (②). When the update counter reaches the batch size (③), HBalloc first writes an undo log entry to the log region in PM (④) and then flushes the metadata to the chunk header in PM (⑤), making the updates persistent. But if the system crashes before the periodic metadata synchronization, HBalloc rolls back all allocation and deallocation operations after last synchronization point.

We call an object “activated” once HBalloc persists its allocation operation. To avoid the corner case that an activated object references an unactivated object, we provide `HBalloc_activate()` for the user to explicitly activate an allocated object. This function flushes the vheader of the given object to PM immediately to activate this object, prior to the periodic synchronizations determined by the batch size. Through this function, the user can make sure that certain object is activated before it is referenced by another. Section V-A shows an example of how to use this function to guarantee the consistency of a program.

Batched persistence shares the same methodology to eventual consistency models [18], [20] by updating the consistent state lazily. It jumps from one consistent state to another periodically, avoiding falling into any inconsistent state. The batch size determines the frequency of synchronization and the runtime performance. In the corner case, when the batch size is set to 1, HBalloc reverts to a normal PM allocator using direct persistence. Conversely, when the batch size is set to $+\infty$, HBalloc reverts to a DRAM allocator without persistence. Batched persistence mechanism redirects the PM writes to DRAM, which not only eliminates latency increases caused by repeated PM flushes but also avoids the read-write interference [1] in PM.

D. Position-Independent Persistent Pointer

To break the limitation of offset-based persistent pointers and support object migration, we propose PIPs. A PIP is a 64-bit integer consisting of a 32-bit timestamp (in seconds) and a 32-bit object counter. Once an object is allocated, HBalloc increases the object counter by one via the CAS instruction. HBalloc maintains a hash table to store the mappings between PIPs and virtual addresses of corresponding objects. After allocating an object and assigning it a PIP, HBalloc inserts the $\langle \text{PIP}, \text{address} \rangle$ pair into the hash table. To dereference a PIP, HBalloc looks up the hash table and returns its current virtual address. If an object moves from one position to another, HBalloc will immediately update the virtual address in the hash table. Once the system remaps the persistent heap, HBalloc updates all virtual addresses in the hash table by subtracting the base address of last mapping and adding that of current mapping. HBalloc guarantees the consistency of this hash table by version numbers and undo logs.

Defragmentation requires merging fragmented memory blocks, which often involves relocating objects that may be referenced. For example, in a persistent hash table, deletions can leave gaps between live objects. By using offset-based pointers (OPs), such compaction becomes infeasible—moving

an object X invalidates all existing pointers to it. This limitation arises because offsets are location-dependent, and there is no global mechanism to identify and update all references to X after it moves. Unlike the existing designs, HBBalloc leverages PIPs to enable safe object relocation. When object X is moved, only its virtual address in the central PIP table needs to be updated. Since all references to X go through this PIP table, no other pointers require modification.

A PIP is used only as a persistent reference to an allocated object. HBBalloc records the mapping between the PIP and the corresponding virtual address in the persistent PIP-to-VA table. Before an object is accessed, the application translates the PIP into a transient virtual address through this table. Applications perform subsequent memory accesses and pointer arithmetic on the translated virtual address, rather than on the PIP value itself. Therefore, HBBalloc does not require redefining pointer arithmetic on persistent references.

E. NUMA-Aware Allocation and Deallocation

Modern persistent-memory platforms expose multiple NUMA domains, where each CPU socket has lower latency access to its local persistent-memory device and higher latency access to remote devices [16]. The original HBBalloc design manages PM through a single heap space and shared allocator metadata. Although this design is simple and easy-to-use, it cannot distinguish whether an allocation request is served from local or remote PM. As a result, when threads run on different sockets, allocator metadata and persistent objects may be frequently accessed across NUMA nodes, which significantly reduce allocation throughput.

To address this problem, we design a NUMA-aware extension for HBBalloc to favor NUMA-local allocation while preserving its nonblocking design. The extension is built around HBBalloc's persistent-pointer abstraction and region/chunk hierarchy. Each NUMA node owns an independent persistent heap, and the heap header includes each heap's NUMA node identifier. Instead of exposing these heaps directly to applications, HBBalloc hides them behind its persistent-pointer routing layer. A persistent pointer encodes the resident heap node and an offset within that heap. Thus, HBBalloc can translate the pointer to the correct virtual address. This preserves HBBalloc's original allocation interface while enabling local-first allocation and remote fallback across all NUMA nodes.

NUMA-Aware Heap, Region, and Tcache: HBBalloc manages memory in regions and chunks. In the NUMA-aware extension, HBBalloc divides the global region space into per-node region pools. Each NUMA node owns an independent region pool backed by its local persistent heap. A heap header records the NUMA node to which the region belongs, and allocation threads preferentially acquire regions from their local node. Only when the local region pool cannot satisfy the request does HBBalloc fall back to regions from another NUMA node.

HBBalloc uses thread-local caches to reduce synchronization overhead during small allocations. However, a single global tcache array may still cause remote metadata accesses when threads from different sockets compete for allocator metadata in the same persistent heap. To address this problem, we partition the tcache space by NUMA nodes. Each heap maintains its own tcache area, and each tcache records the NUMA node it belongs to. When a thread first enters the allocator,

HBBalloc determines its local NUMA node and caches this value in thread-local storage. The subsequent allocation and deallocation operations reuse the cached local-node information instead of invoking `getcpu()` on every allocator operation, which introduces extra system-call overhead. After the local node is determined, the thread searches for its tcache only in the tcache array of that node. If no tcache has been assigned to the thread, HBBalloc occupies a free tcache entry from the same node and binds it to the thread. When the tcache needs to be refilled, HBBalloc obtains chunks from the local node's region pool. Therefore, both the tcache metadata access and the persistent object placement follow the same NUMA local first policy.

NUMA-Aware Logging, Allocation, and Deallocation: HBBalloc uses undo logs to make allocator metadata updates recoverable. In the NUMA-aware extension, we replace the single global log with per-heap undo logs. Thus, each persistent heap has its own log. A thread writes metadata updates to the log of its local heap, where its tcache and region metadata are normally located. This keeps logging local in the common case and avoids contention on one shared persistent log.

For an allocation request, HBBalloc first reads the thread's cached local node from thread-local storage. If this value has not been initialized, HBBalloc determines the NUMA node of the CPU currently executing the thread and stores it as the thread's local node. The allocator then selects the local heap of this node and looks up the thread's tcache in the tcache area of local heap. If the requested size class has available blocks in the tcache, HBBalloc returns one block directly. Otherwise, HBBalloc refills the tcache by acquiring chunks from a region. During refill, HBBalloc tries to occupy a free region from the thread's local heap. If no region in the local heap can be occupied, HBBalloc repeats the same occupation attempt on the other nodes and uses the first successfully occupied remote region to refill the tcache.

For deallocation, HBBalloc uses the thread's cached local node to select its local heap, tcache, and undo log. The allocator translates the persistent pointer to a virtual address through HBBalloc's persistent-pointer translation layer. In the offset-based pointer representation, the pointer contains the resident heap node and an offset within that heap; in the PIP configuration, the pointer is resolved through the PP-to-VA table. After obtaining the virtual address, HBBalloc identifies whether the object belongs to one of the NUMA-aware heaps by checking whether the address falls within any recorded heap range. Small objects are returned through the tcache path, while large objects are released through the large-allocation path. Metadata updates are written to the undo log of the thread's local heap.

V. IMPLEMENTATION

We implement HBBalloc as an easy-to-use user-level programming library that is independent of specific PM platforms, since we never use any third-party library except for the standard `mmap()` system call [9], [37]. HBBalloc is built on top of the EXT4-DAX file system and can be easily transplanted to other DAX-enabled systems.

A. Programming Model

HBBalloc's API is fully compatible with PMDK [42] and other memory allocators. Programmers simply need to

replace the initialization call with `HBalloc_start()`, allocation with `HBalloc_malloc()`, and deallocation with `HBalloc_free()`. Almost all allocators follow this standard interface pattern, except for Makalu [3], which uses a two-phase allocation scheme. HBalloc provides eight functions for developers to exploit the persistent heap. Developers initiate an HBalloc instance by `HBalloc_start()`. HBalloc opens the specified persistent heap file and maps it to the virtual address space. Developers then use `HBalloc_malloc()` or `HBalloc_free()` to allocate or reclaim objects. Finally, developers use `HBalloc_exit()` to safely exit the HBalloc instance. Besides, HBalloc provides the `HBalloc_activate()` for users to explicitly activate an object to prevent it from being rolled back upon a crash. The `HBalloc_activate()` can also activate all allocated objects at once if it gets an empty persistent pointer (NPP). The `HBalloc_persist()` and `HBalloc_barrier()` guarantee to persist a given region in the appropriate order, similar to the CLWB and SFENCE instructions [42]. Finally, the `deref()` is used to dereference a persistent pointer.

Fig. 6 shows an example of constructing a persistent linked list using HBalloc. To use HBalloc library, we include the header file “HBalloc.h” (Line 1). We start an HBalloc instance by specifying the path and size of the persistent heap file (Line 4). We allocate an object as the head node and activate it (Lines 5–7). We dereference the persistent pointer to the head node and set its linked pointer as empty (Line 9). The head node is guaranteed to be persisted after `HBalloc_persist()` and `HBalloc_barrier()` (Lines 10 and 11). Subsequently, we allocate ten new nodes and insert them into the linked list one by one (Lines 12–23). Each of the new node is activated and persisted to guarantee the consistency of the linked list. Finally, we exit the HBalloc instance safely with `HBalloc_exit()` (Line 24).

B. Memory Management

Excluding the heap header that stores the global metadata of the persistent heap, HBalloc logically partitions the persistent heap into independent regions, each 256 MB in size, to enable nonblocking global allocation. Our 256-MB regions are significantly larger than typical allocation sizes (99% of requests are ≤ 8 kB in benchmarks in Fig. 2). For oversized requests (> 256 MB), HBalloc falls back to `mmap()`, thereby avoiding artificial fragmentation. HBalloc manages each region independently in units of chunks. Each region contains a header that stores the core metadata of this region (e.g., the number of free chunks in this region and the address of the first free chunk) and the remaining part is divided into chunks, each of which is 16 kB in our implementation. Chunks are the basic units of global allocation. Free chunks in a region are linked into a list ordered by addresses. HBalloc returns one or more chunks from the free chunk list when a global allocation is assigned to this region. The free chunk list per region ensures that contiguous allocations are possible.

Chunks in a tcache are linked into multiple lists for different size classes. When a chunk is allocated from the global persistent heap to replenish an exhausted tcache, it will be added to the chunk list of the corresponding size class in the tcache and divided into some small blocks. The size of each small block is determined by the size class as well. Free or

```

1 #include <HBalloc.h>
2 typedef struct {int data; PP_t next;} Node;
3 int main(void) {
4     HBalloc_start("/mnt/pmem/pool", 4*1024*1024*1024);
5     PP_t head = HBalloc_malloc(sizeof(Node));
6     HBalloc_activate(head);
7     HBalloc_barrier();
8     deref(head)->data = -1;
9     deref(head)->next = NPP;
10    HBalloc_persist(head, sizeof(Node));
11    HBalloc_barrier();
12    for (int i = 0; i < 10; i++) {
13        PP_t new_node = HBalloc_malloc(sizeof(Node));
14        HBalloc_activate(new_node);
15        HBalloc_barrier();
16        deref(new_node)->data = i;
17        deref(new_node)->next = deref(head)->next;
18        HBalloc_persist(new_node, sizeof(Node));
19        HBalloc_barrier();
20        deref(head)->next = new_node;
21        HBalloc_persist(head, sizeof(Node));
22        HBalloc_barrier();
23    }
24    HBalloc_exit();
25    return 0;
26 }
```

Fig. 6. Example of HBalloc programming.

allocated blocks in a chunk are marked by a bitmap that is stored in the chunk header. The chunk header also contains the size class of this chunk, the link pointer to next chunk, the number of free blocks and the index of the first free block in this chunk, and the pointer to the vheader of this chunk. Each chunk will be assigned a vheader in DRAM once the chunk is allocated, to enable the batched persistence mechanism. The vheader will be destroyed once the chunk is freed.

C. Allocation and Reclaim

Once the `HBalloc_malloc()` is invoked, HBalloc first checks the given allocation size. If the size does not exceed 8 kB (the maximum size class in our implementation), it will be aligned up to the nearest standard size class. HBalloc then searches for free blocks in the corresponding size class in its tcache. If the tcache is exhausted and no free block is found, HBalloc will allocate a chunk from the global persistent heap to replenish the tcache. Finally, HBalloc allocates and returns a free block from the tcache. For allocations larger than 8 kB, HBalloc aligns the allocation size up to the multiple of 16 kB (the size of each chunk) and directly allocates some contiguous free chunks from the global persistent heap. For example, an allocation request of 40 kB will obtain free space consisting of three contiguous chunks, the size of which is 48 kB. If the allocation size is even larger than 256 MB (the size of each region), HBalloc directly memory-maps new space by invoking the `mmap()` system call.

The reclaiming process corresponds to the allocation process one by one. If the size of the reclaimed object is no larger than 8 kB, the object will be returned to the corresponding size class in the tcache. Otherwise, it will be directly returned to the global persistent heap (for objects no larger than 256 MB) or to the operating system (for objects larger than 256 MB). Free chunks in a tcache are finally returned to the global persistent heap when the corresponding thread exits or the number of free chunks exceeds the tcache size, which is defined as the maximum number of free chunks that a tcache can hold.

D. Crash Consistency and Recovery

HBalloc employs batched persistence mechanism to reduce the high overheads of repeated cacheline flushes, without sacrificing the crash consistency. Once a chunk is allocated, HBalloc creates a vheader in DRAM and copies the chunk header to the vheader. Subsequent allocations from this chunk only modify the vheader in DRAM, which would be completely lost after a crash, while the chunk header in PM still keeps the consistent state. The vheader will be flushed back to PM to synchronize the updates when the update counter reaches the batch size or the user invokes `HBalloc_activate()`. Write-ahead logs (WALs) guarantee “all or nothing” synchronization. Upon a synchronization, HBalloc first writes the vheader to a dedicated log file and persists it. HBalloc then marks the transaction as `RUNNING` and starts to synchronize the vheader to the corresponding chunk header. After finishing the synchronization and persisting the chunk header, HBalloc marks the transaction as `COMMITTED`, which means the synchronization process is completed. However, if a crash occurs before the synchronization is completed, the system may persist only part of the chunk header, leading to inconsistent state of metadata. After reboot, HBalloc will scan the log file and find out uncommitted transactions remaining at the `RUNNING` state. HBalloc further restores the heap metadata by rolling back these uncommitted transactions, ensuring the consistency of metadata. The function `HBalloc_activate()` enables users to explicitly activate an object by flushing the corresponding vheader of the given object to PM. This process is also atomic with WALs.

Except for updating the chunk header, allocating chunks from global persistent heap also needs to utilize WAL to guarantee the crash consistency. Since global allocations need to allocate a chunk from the global free chunk list and insert it into the appropriate chunk list in a tcache, multiple link pointers would be modified. If a crash occurs before all the link pointers are modified and persisted, the crash may corrupt the chunk list, resulting in permanent memory leak. To avoid this problem, HBalloc coalesces these pointers together into a log entry, which will be written to the log file before the actual modification of pointers. Hence, HBalloc could restore these link pointers based on the undo log, avoiding dangling pointers and memory leak.

Recovery Procedure and Guarantees: Upon restart, HBalloc maps the persistent heap and the undo-log file before serving new allocation requests. Each thread owns a log slot. Before HBalloc modifies allocator metadata, it records the old metadata values in this log slot, so that recovery can rollback an interrupted update. HBalloc scans the log slots whose state is `LOG_RUNNING`. For each such slot, it rolls back the interrupted metadata update by copying the logged old values back to the corresponding persistent metadata locations. For a vheader synchronization, HBalloc restores the persistent chunk-header fields from the undo log. For chunk-list updates, HBalloc restores the affected chunk-list metadata, including the modified chunk header and neighboring list pointers.

After unfinished updates are rolled back, HBalloc updates persistent virtual-address references according to the heap mapping address of the current execution. In particular, it adjusts the link pointers stored in region chunk lists and tcache chunk lists and resets thread ownership fields in tcaches.

Therefore, after recovery, allocator metadata is returned to a state prior to the interrupted metadata transaction.

VI. PERFORMANCE EVALUATION

A. Experimental Configurations

Platform: We evaluate HBalloc on a Linux server (Ubuntu 18.04) equipped with 6×128 -GB Intel Optane DC PMM 100 Series and 12×16 -GB DDR4 DRAM. There are two CPU sockets in the server and each socket is equipped with a 26-core Intel Xeon Gold 6230R at 2.10 GHz. We connect the Optane DC PMMs to the same CPU socket and configure them in the interleaved and App Direct Modes [42].

Benchmarks: We utilize four well-recognized and widely used benchmarks for extensive evaluation. *Threadtest* [2] is a basic benchmark for multithreading performance, which allocates and reclaims a large number of objects concurrently. *Shbench* [33] is a stress test for allocators, which allocates objects of random sizes. *Larson* [28] simulates a scenario in which some of the objects allocated by one thread are left to be reclaimed by another. *Prod-con* [43] is a classic producer-consumer pattern where objects are allocated by producers and reclaimed by consumers concurrently.

Configurations: In our evaluation, the region size of HBalloc is set as 256MB and the tcache size is set as 8 bytes by default. In small allocation tests, we evaluate HBalloc with both *batchsize* = 1 (HBalloc-1) and *batchsize* = 8 (HBalloc-8) to demonstrate the improvements brought by the batched persistence mechanism. Large allocations are not affected by the batch size, so all experiments use *batchsize* = 8 by default, except where noted (e.g., HBalloc-1 in Figs. 7–9). This default batch size is empirically optimal, as shown in Fig. 10(b). Moreover, all experiments leverage PIP pointers by default, since PIP is the default configuration in HBalloc and a key innovation enabling object migration, unless explicitly stated (e.g., Fig. 10, which compares PIP and OP).

We compare HBalloc with the state-of-the-art log-based PM allocators, including libpmemobj of PMDK [12], [42], *nvm_malloc* [44], *NVAlloc-log* [15], and *PMA* [49]. Other garbage collection (GC)-based allocators, such as Makalu [3] and *Ralloc* [4], are not included to facilitate fair comparison, since their designs rely on offline GC instead of write-ahead-logs (WALs). These allocators perform whole-heap traversal to reclaim memory, which introduces different types of overheads and performance tradeoffs, making direct comparison with WAL-based systems inappropriate.

B. Results and Analysis

1) Throughputs of Microbenchmarks: Fig. 7 shows the throughputs of the four benchmarks with small allocation sizes across different PM allocators. HBalloc outperforms its competitors in all four benchmarks. Specifically, HBalloc-8 achieves up to $435\times$, $42\times$, $445\times$, and $23\times$ higher throughputs compared with PMDK, *nvm_malloc*, *NVAlloc-log*, and *PMA*, respectively. HBalloc achieves these improvements mainly because our batched persistence design, which significantly reduces the number of CLWB instructions. Nearly 87.5% of PM writes are eliminated when we set the batch size to 8, which brings up to $7.8\times$ speedup compared with HBalloc-1. In large-size allocation, HBalloc also outperforms other allocators and achieves better scalability. As shown in Fig. 8,

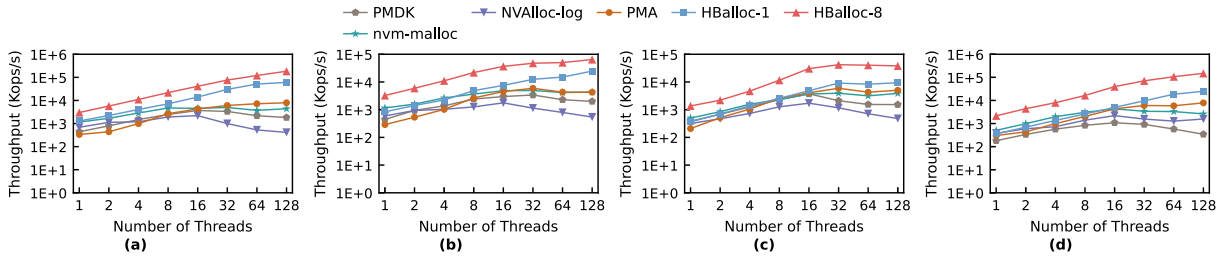


Fig. 7. Throughputs (log10 scaled) of small allocations across different PM allocators. (a) Threadtest. (b) Shbench. (c) Larson. (d) Prod-con.

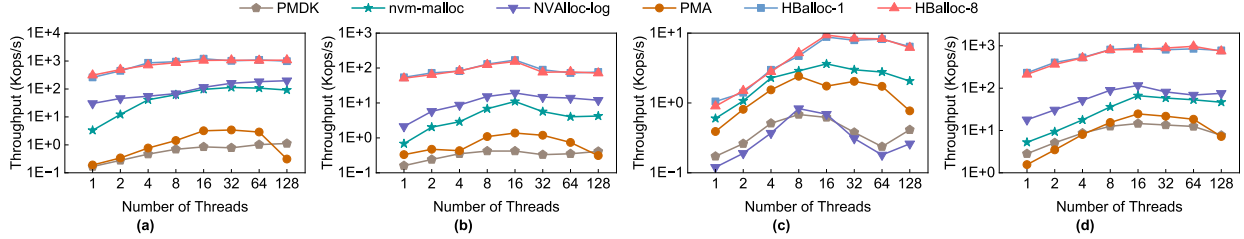


Fig. 8. Throughputs (log10 scaled) of large allocations across different PM allocators. (a) Threadtest. (b) Shbench. (c) Larson. (d) Prod-con.

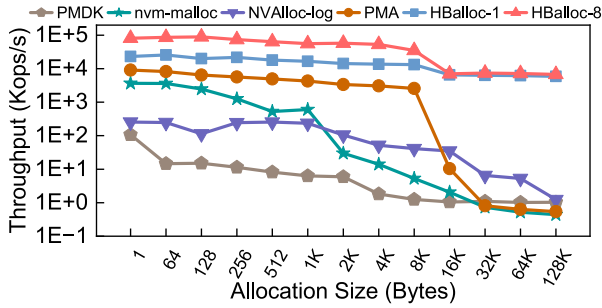


Fig. 9. Throughputs (log10 scaled) of various allocation sizes across different PM allocators.

HBalloc achieves up to $151\times$, $40\times$, $15\times$, and $160\times$ higher throughputs compared with PMDK, `nvm_malloc`, `NValloc-log`, and `PMA`, respectively. Benefiting from our nonblocking design, HBalloc significantly reduces the overhead of concurrence control in global allocations. In contrast, other lock-based allocators suffer from high contentions in global allocations, and their throughputs even decrease when the number of working threads increases.

In addition to the four benchmarks, we further evaluate the performance of HBalloc for various allocation sizes. As shown in Fig. 9, when the allocation size increases from 1 B to 128 kB, the throughputs of PMDK, `nvm_malloc`, `NValloc-log`, and `PMA` drop by $103\times$, $8361\times$, $207\times$, and $16944\times$, respectively, whereas the throughput of HBalloc only drops by $13\times$. The performance degradation of PM allocators typically occurs due to lower tcache hit ratios, especially when the allocation size surpasses the maximum size class. Large allocations are hence directly allocated from global persistent heap, causing high overheads of concurrence control. Our nonblocking global allocation design effectively addresses this problem and mitigates the performance degradation of HBalloc. When the allocation size surpasses the maximum size class, the performance degradation of HBalloc is $8\times$, $103\times$, $14\times$, and $240\times$ slower than PMDK, `nvm_malloc`, `NValloc-log`, and `PMA`, respectively, exhibiting much better performance stability.

2) Recovery Time of Different Allocators: To evaluate the recovering efficiency of HBalloc, we initiate an HBalloc instance to allocate 10^8 64 B objects with 128 working threads and kill the process suddenly before it finishes the job and normally exits. This triggers the recovery routine upon restart since the HBalloc instance does not invoke the `HBalloc_exit()`. As a result, HBalloc successfully recovers 128 uncommitted transactions and rolls back 1024 unactivated objects within 29 ms. `Nvm_malloc`, `NValloc-log`, and `PMA` finish the same recovery process within 34, 42, and 27 ms, respectively.

3) Throughputs Under Different Parameters: To investigate the effects of different parameters on the performance of HBalloc, we run threadtest and shbench with 128 working threads under various region sizes, batch sizes, and tcache sizes.

Fig. 10(a) shows that the throughputs first increase and then decrease, when the region size grows from 1 MB to 8 GB. This behavior is attributed to the interplay among region sizes, rehashings, and hash collisions. When the region size is too small, regions are quickly exhausted, thus leading to frequent rehashings, which distribute working threads to new regions but introduce substantial time overhead and thus reduce the overall performance. On the contrary, when the region size is too large, the reduced number of regions increases the probability of hash collisions, hence negatively impacting the overall performance. The optimal balance point is observed at 256 MB, where the throughput peaks due to the minimized impact from both rehashings and hash collisions.

Fig. 10(b) illustrates the throughput changing trend with increasing batch sizes. We observe that the larger batch size results in higher throughput, due to fewer PM writes. However, the growth rate of throughput is initially rapid, but then slows down, and ultimately stabilizes toward an upper limit. Therefore, larger batch size is not always better, since it will compromise the persistence of the allocator. Based on the results in Fig. 10(b), 32 is deemed an appropriate value for the batch size.

We investigate the performance of HBalloc by examining various tcache sizes, as shown in Fig. 10(c). The tcache size is

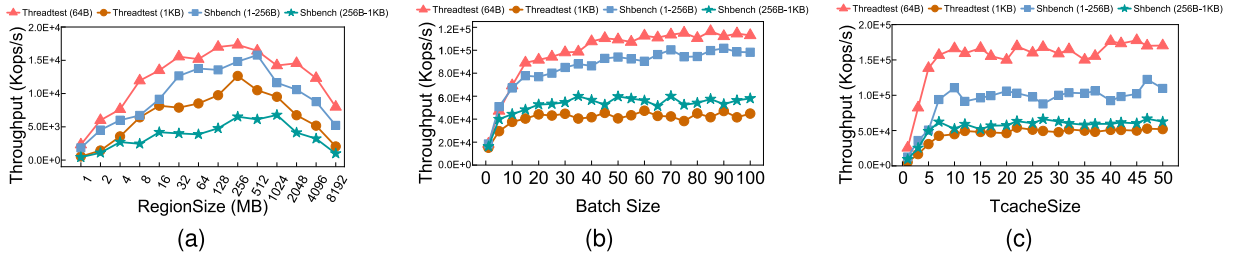


Fig. 10. Throughputs of HBallloc under different region sizes, batch sizes, and tcache sizes. (a) Throughputs of various region sizes. (b) Throughputs of various batch sizes. (c) Throughputs of various tcache sizes.

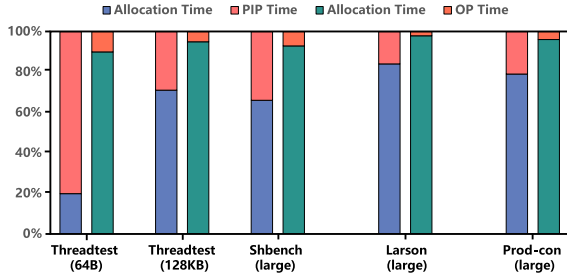


Fig. 11. Persistent pointer dereference overheads.

defined as the maximum number of free chunks that a tcache can hold. If the number of free chunks in a tcache exceeds the tcache size, these free chunks will be returned to the global persistent heap. Therefore, when the tcache size is small, the tcache will be frequently flushed and filled, thus leading to frequent global allocations, and the throughput becomes low. With the growth of the tcache size, the throughput gradually increases due to higher tcache hitting rate, but it ultimately stabilizes toward an upper limit. Based on the experimental results in Fig. 10(c), we set the tcache size to 8 as the default.

4) Time Costs of Dereferencing Persistent Pointers: We evaluate the time overheads of our proposed PIP and traditional offset-based persistent pointer (OP) by running threadtest-64B, threadtest-128KB, shbench-large, larsen-large, and prod-con-large. Fig. 11 shows the percentage of time dedicated to memory allocation and pointer translation for PIP and OP. PIP accounts for 80%, 29%, 34%, 16%, and 21% of the execution time in threadtest-64B, threadtest-128KB, shbench-large, larsen-large, and prod-con-large, respectively, whereas OP accounts for 10%, 5%, 7%, 2%, and 4%, respectively. The PIP produces higher time overheads compared with OP and even dominates the execution time in threadtest-64B, since we have to maintain a map of PIPs and virtual addresses. Threadtest-64B frequently allocates and frees many small objects, leading to frequent insertion and deletion of the map, thus dominating the execution time. Therefore, we provide developers with a flexible choice of pointers in HBallloc. Based on their demands, developers can choose to use PIP, OP, or virtual address by changing macro definition in HBallloc. If the allocated objects do not need to be moved, OP is recommended. Otherwise, PIP is the only choice. Moreover, developers can also choose to simply use virtual address, if the persistent heap is guaranteed to be mapped to the same address or does not need to be remapped at all. We leave the optimization of PIP as our future work.

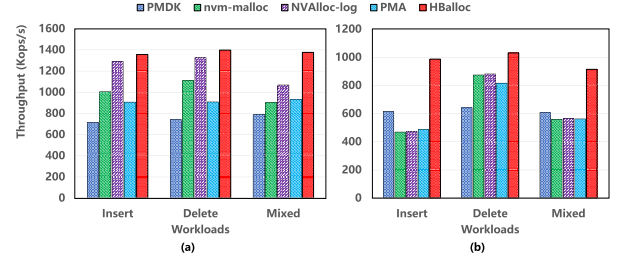


Fig. 12. Throughputs of FPTree with different PM allocators. (a) FPTree-4. (b) FPTree-1024.

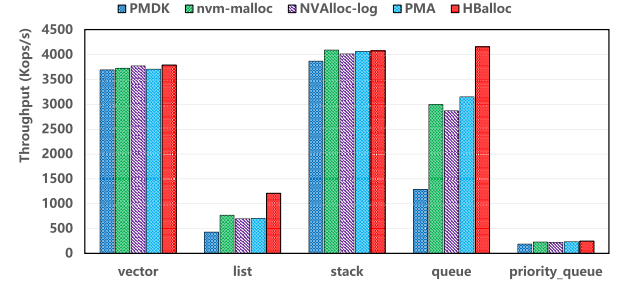


Fig. 13. Throughputs of persistent data structures with different PM allocators.

C. Analysis of NUMA Effects

We evaluate the NUMA effects of persistentmemory allocators on a dual-socket machine, and each socket is equipped with 2×128 PMEM devices. The evaluated allocators include NVALloc [15], PMAlloc [16], the original HBallloc, and the NUMA-aware HBallloc. To expose NUMA effects, our experiments explicitly distribute threads across both sockets. Specifically, even-numbered threads are assigned to socket 0 and odd-numbered threads are assigned to socket 1. Fig. 14 shows the throughputs of NVALloc, PMAlloc, HBallloc, and NUMA-aware HBallloc. For Threadtest, Shbench and Larson, NUMA-aware HBallloc achieves high performance by up to $1.5 \times$ over the native HBallloc and more than $11 \times$ over PMAlloc. These workloads frequently access allocator metadata, tcaches, and persistent heap structures. By assigning each thread to its local NUMA heap and localizing metadata and log accesses, NUMA-aware HBallloc reduces remote heap accesses. For Prod-con, since allocators always allocate local memory blocks, NUMA-aware HBallloc achieves similar throughput to HBallloc. Moreover, HBallloc achieves the performance by up to $17.1 \times$ over PMAlloc at 128 threads, since allocator already benefits significantly from HBallloc's non-blocking design. Nevertheless, NUMA-aware HBallloc still provides gains for Threadtest, Shbench and Larson, confirming that local-first heap space, allocator metadata, and persistence logs improve performance by reducing remote NUMA

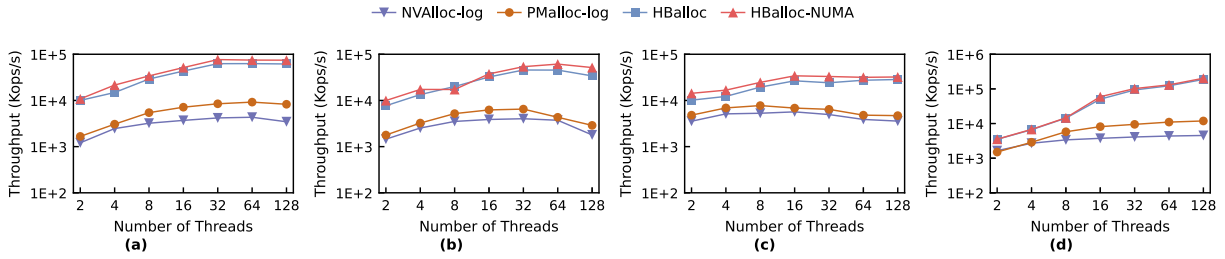


Fig. 14. Throughputs on the dual-socket configuration with NUMA effects. (a) Threadtest. (b) Shbench. (c) Larson. (d) Prod-con.

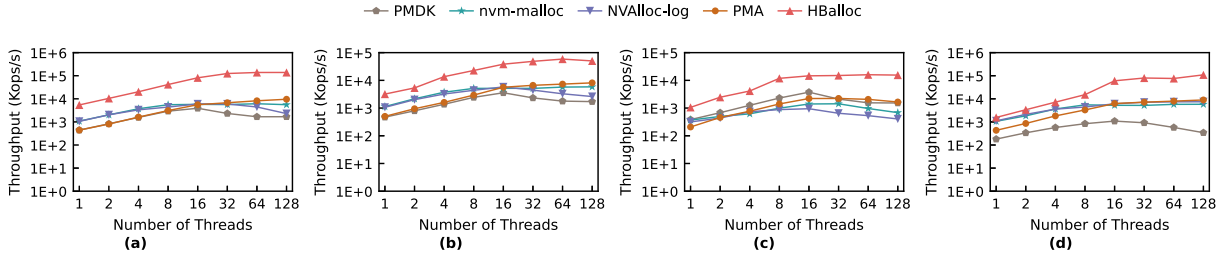


Fig. 15. Throughputs on a single socket configuration. (a) Threadtest. (b) Shbench. (c) Larson. (d) Prod-con.

accesses. To isolate HBalloc’s gains from NUMA effects, we additionally evaluate all allocators under a single-socket configuration, where all benchmark threads are pinned to socket 0. As shown in [15], HBalloc outperforms PMDK, nvm-malloc, NValloc-LOG and PMA across all workloads and thread counts, achieving up to $319\times$, $24.8\times$, $60.4\times$, $18.4\times$ higher throughputs, respectively. Since all threads run on the same socket, the observed performance advantage stems from HBalloc’s non-blocking allocation mechanism, lightweight tcache-based fast path and batched persistence design, which reduce synchronization overhead and metadata contention.

D. Real-World Applications

We integrated HBalloc into the FPTree [36], a real-world key-value store application. It is a persistent B+tree with inner nodes in DRAM and leaf nodes in PM. In order to evaluate the performance of FPTree with different PM allocation schemes, we replace the default PM allocation module in FPTree with our HBalloc and other state-of-the-art PM allocators. We evaluate the performance of FPTree with insert, delete and mixed workloads (50% insert and 5% delete). We evaluate both the 4th-order FPTree (size of each node is 128B) and the 1024th-order FPTree (size of each node is 32KB) to measure small and large allocations, respectively. As shown in Fig. 12, compared to PMDK, nvm_malloc, NValloc-log and PMA, the 4th-order FPTree with HBalloc yields up to 74%, 52%, 30% and 47% higher throughputs, while the 1024th-order FPTree with HBalloc yields up to 50%, 64%, 62% and 63% higher throughputs. The lookup operation in FPTree is not evaluated because it does not involve memory allocations. Furthermore, we implemented a persistent version of C++ STL with HBalloc, including the vector, list, stack, queue and priority queue. The vector and stack are implemented as dynamic arrays, while the list, queue and priority queue are implemented as doubly linked lists. All of these data structures are guaranteed to be crash-consistent with writeahead-logs. We integrated different PM allocators into these data structures and evaluated their performance with an update heavy workload (50% push and 50% pop). Fig. 13 shows that compared to state-of-the-art

PM allocators, HBalloc delivers up to $1.03\times$, $2.82\times$, $1.05\times$, $3.24\times$ and $1.32\times$ higher throughputs in the vector, list, stack, queue and priority queue, respectively. It is easy understanding that linked list-based data structures, like list and queue, can gain more improvements from HBalloc, since they need to allocate memory more frequently. Each time they create a new node, they need to allocate memory. On the other hand, the performance of dynamic array-based data structures like vector and stack is not heavily affected by the allocators since they do not need to allocate memory frequently. They only need to allocate memory when expanding the array.

E. DRAM-SSD Platform

To efficiently and quantitatively demonstrate that HBalloc is independent of Intel Optane PM, we conduct experiments on another persistent platform that consists of typical DRAM and SSD devices. In this DRAM-SSD platform, the DRAM contributes to high performance and SSD offers nonvolatility. We create a huge file (128 GB) on the SSD to simulate other forms of PM or NVM devices. The file is then memory-mapped using the `mmap()` system call [9], [37] to serve as the persistent heap. We run the same benchmarks as in Section VI-B1, and the evaluation results are shown in Figs. 16 and 17. This emulation is reasonable since any PM or NVM device can be regarded as a file in Linux system due to the “everything is a file” concept [46]. The file abstraction hides the underlying hardware details and makes the PM or NVM platform transparent to users.

VII. DISCUSSION

HBalloc reduces PM writes and eliminates repeated logs and cacheline flushes through batched persistence mechanism. The main idea is to lazily and atomically update the heap metadata [18], [20]. The batched persistence mechanism does guarantee the crash consistency of the persistent heap by enforcing the metadata jumping from one consistent state to another. However, delaying the persistence of heap metadata also creates the potential for rolling back multiple allocated objects before they are activated, which introduces some

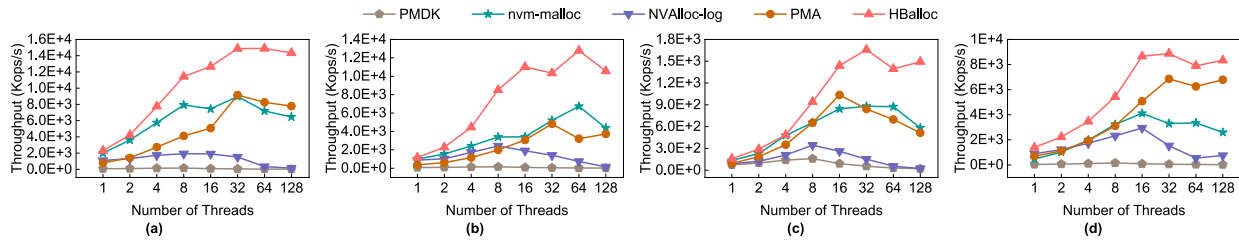


Fig. 16. Throughputs of small allocations on the DRAM-SSD platform. (a) Threadtest. (b) Shbench. (c) Larson. (d) Prod-con.

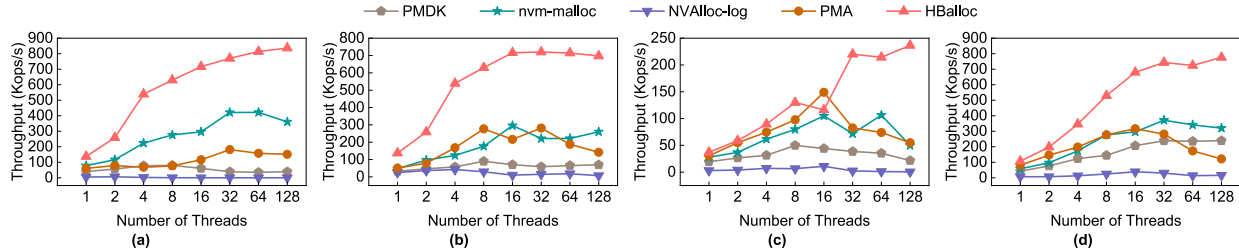


Fig. 17. Throughputs of large allocations on the DRAM-SSD platform. (a) Threadtest. (b) Shbench. (c) Larson. (d) Prod-con.

complexity in writing a consistent program. We have realized this problem and provided two approaches for users to address it. One is enabling the users to activate objects via the `HBBalloc_activate()` and the other is simply setting the batch size to 1. In fact, this problem is not unique to HBBalloc, and it is common in the PM-related designs [42]. For example, the developers have to carefully flush back some cachelines via the CLWB instruction and organize the persisting order via the SFENCE.

The HBBalloc proposes position-independent persistent pointers to support object migrations, mainly for supporting memory space defragmentation in our future works. In large allocations, some contiguous free chunks need to be allocated, but if the persistent heap is too fragmented to find enough contiguous free memory space, the allocation would fail, even if more than 80% of memory space in the persistent heap is idle, which causes severe memory waste. However, the PIPs only resolve the pointer-update issue caused by moving persistent objects. A practical defragmentation system also requires policies to trigger relocations, mechanisms for moving fragmented regions/objects and supporting crash consistency.

HBBalloc focuses on ADR mode rather than eADR. This is because many real-world PM systems rely on explicit cacheline flushes for correctness. While eADR simplifies persistence by eliminating the need for CLWB instructions, HBBalloc's design targets on the more general ADR case to ensure broad applicability. The recovery model uses a WAL to track metadata changes, enabling crash consistency by rolling back incomplete transactions during recovery. The `HBBalloc_activate()` function provides explicit durability control but is optional—without it, unactivated allocations may roll back after crashes, while activated ones guarantee persistence.

Recent whole-system persistence (WSP) proposals [54], [55], [59] adopt a persistence model that transparently preserves and restores application state across failures. On the other hand, the allocator-level persistent-heap systems are object-centric, exposing durable objects whose allocation, reclamation, and lifetime are explicitly managed by applications. As a result, allocator-level persistent-heap approaches remain necessary in application domains that organize data as

long-lived persistent objects, such as persistent indexes and key-value stores [4], [10], [51], where efficient object allocation, space reuse, metadata management, and fragmentation control are fundamental requirements independent of crash-consistency mechanisms. HBBalloc targets this persistent-heap programming model.

VIII. RELATED WORK

DRAM Allocators: Dynamic memory allocator for DRAM has been investigated for decades. Hoard [2] is an efficient and highly scalable allocator that has been widely used in various platforms. It avoids false sharing and achieves good scalability for multithreaded programs simultaneously. Jemalloc [19] is an important allocator for FreeBSD to provide scalable concurrent allocation for multiprocessor computer systems. TCMalloc [21] is a commercial allocator developed by Google, which employs thread-local memory cache to achieve high performance. There are some optimization schemes [29] for TCMalloc as well. LRMalloc [30] is a modern lock-free DRAM allocator that leverages atomic instruction to achieve high-efficiency concurrence control.

Log-Based Persistent-Memory Allocators: Different from DRAM allocators, PM allocators need to guarantee the crash consistency of metadata, preventing metadata from corruption when a system failure occurs. Write-ahead log is employed in many PM allocators to achieve this goal, such as Poseidon [17], `nvm_malloc` [44], `PAllocator` [35], `NValloc-log` [15], and `PMA` [49]. Log-based PM allocators roll back metadata to the nearest consistent state after a crash.

Garbage-Collection-Based Persistent-Memory Allocators: To reduce the overheads of writing logs and flushing metadata, some modern PM allocators, such as Makalu [3], `ralloc` [4], and `NValloc-GC` [15], employ offline GC to relax the online constraints of heap metadata persistence. They typically generate some root objects at dedicated positions of the persistent heap to trace all live user objects and traverse the persistent heap from the persistent root objects during GC. Any object that is unreachable from the root object will be reclaimed. The GC-based PM allocators eliminate the overhead of write-ahead log but introduce the overhead of GC, since they need to traverse the whole persistent heap.

IX. CONCLUSION

In this article, we present three challenges of PM allocators, including the size sensitivity that causes performance degradation in large allocations, the high persistence tax that comes from the overheads of repeated logging and cacheline flushes, and the limitation of offset-based persistent pointers. To address these problems simultaneously, we propose a cost-efficient PM allocator, called HBalloc, by leveraging nonblocking global allocations, batched persistence mechanism, and position-independent persistent pointers. We evaluate HBalloc with both microbenchmarks and real-world applications. Extensive experimental results demonstrate that HBalloc outperforms the state-of-the-art PM allocators in both small and large allocations. Moreover, HBalloc is independent of specific platforms and can be easily coalesced with other frameworks. We have released the open-source codes of HBalloc in GitHub for public use.

ACKNOWLEDGMENT

The authors acknowledge that generative AI tool was only used to assist with readability enhancement and code improvements. Its use was only confined to improving presentation clarity and identifying potential coding issues.

REFERENCES

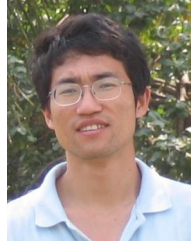
- [1] V. Banakar, K. Wu, Y. Patel, K. Keeton, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "WiscSort: External sorting for byte-addressable storage," *Proc. VLDB Endowment*, vol. 16, no. 9, pp. 2103–2116, May 2023.
- [2] E. D. Berger, K. S. McKinley, R. D. Blumofe, and P. R. Wilson, "Hoard: A scalable memory allocator for multithreaded applications," *ACM SIGPLAN Notices*, vol. 35, no. 11, pp. 117–128, Nov. 2000.
- [3] K. Bhandari, D. R. Chakrabarti, and H.-J. Boehm, "Makalu: Fast recoverable allocation of non-volatile memory," *ACM SIGPLAN Notices*, vol. 51, no. 10, pp. 677–694, Dec. 2016.
- [4] W. Cai, H. Wen, H. A. Beadle, C. Kjellqvist, M. Hedayati, and M. L. Scott, "Understanding and optimizing persistent memory allocation," in *Proc. ACM SIGPLAN Int. Symp. Memory Manage.*, Jun. 2020, pp. 60–73.
- [5] A. Chen, "A review of emerging non-volatile memory (NVM) technologies and applications," *Solid-State Electron.*, vol. 125, pp. 25–38, Nov. 2016.
- [6] C. Chen et al., "OpenEmbedding: A distributed parameter server for deep learning recommendation models using persistent memory," in *Proc. IEEE 39th Int. Conf. Data Eng. (ICDE)*, Apr. 2023, pp. 2976–2987.
- [7] Z. Chen, Y. Lu, F. Yang, Q. Wang, Y. Wang, and J. Shu, "FlatStore: An efficient log-structured key-value storage engine for persistent memory," in *Proc. 25th Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Mar. 2020, pp. 1077–1091.
- [8] Z. Chen, Y. Hua, B. Ding, and P. Zuo, "Lock-free concurrent level hashing for persistent memory," in *Proc. ATC*, 2020, pp. 799–812.
- [9] J. Choi, J. Kim, and H. Han, "Efficient memory mapped file I/O for in-memory file systems," in *Proc. HotStorage*, 2017, p. 5.
- [10] J. Coburn et al., "NV-heaps: Making persistent objects fast and safe with next-generation, non-volatile memories," *ACM SIGPLAN Notices*, vol. 47, no. 4, p. 105, Jun. 2012.
- [11] I. Corporation. (2022). *Intel Reports Second-Quarter 2022 Financial Results*. [Online]. Available: <https://www.intel.com/news-events/press-releases/detail/1563/intel-reports-second-quarter-2022-financial-results>
- [12] I. Corporation. *Persistent Memory Development Kit*. Accessed: Aug. 29, 2026. [Online]. Available: <https://pmem.io/pmdk/>
- [13] I. Corporation. (2018). *3D-XPoint: A Breakthrough in Non-Volatile Memory Technology*. [Online]. Available: <https://www.intel.com/content/www/us/en/architecture-and-technology/intel-micron-3d-xpoint-webcast.html>
- [14] I. Corporation. (2018). *Redis*. [Online]. Available: <https://github.com/pmem/redis/tree/3.2-nvml/>
- [15] Z. Dang et al., "NValloc: Rethinking heap metadata management in persistent memory allocators," in *Proc. 27th ACM Int. Conf. Architectural Support Program. Lang. Operating Syst.*, Feb. 2022, pp. 115–127.
- [16] Z. Dang et al., "PMAlloc: A holistic approach to improving persistent memory allocation," *ACM Trans. Comput. Syst.*, vol. 42, nos. 3–4, pp. 1–52, Nov. 2024.
- [17] A. Demeri, W.-H. Kim, R. M. Krishnan, J. Kim, M. Ismail, and C. Min, "Poseidon: Safe, fast and scalable persistent memory allocator," in *Proc. 21st Int. Middleware Conf.*, Dec. 2020, pp. 207–220.
- [18] A. Demers, D. Greene, C. Hauser, W. Irish, and J. Larson, "Epidemic algorithms for replicated database maintenance," in *Proc. 6th Annu. ACM Symp. Princ. Distrib. Comput. (PODC)*, 1987, pp. 1–12.
- [19] J. Evans, "A Scalable Concurrent malloc (3) Implementation for FreeBSD," in *Proc. Bsdcan Conf.*, vol. 2, no. 3.1, Ottawa, ON, Canada, 2006, p. 5.
- [20] A. Ganesan, R. Alagappan, A. C. Arpaci-Dusseau, and R. H. Arpaci-Dusseau, "Strong and efficient consistency with consistency-aware durability," in *Proc. FAST*, vol. 17, 2021, pp. 1–27.
- [21] S. Ghemawat and P. Menage. *TCMalloc: Thread-Caching Malloc*. Accessed: Aug. 29, 2026. [Online]. Available: <https://goog-perftools.sourceforge.net/doc/tcmalloc.html>
- [22] J. Gu et al., "Piscis: A scalable and efficient persistent transactional memory," in *Proc. ATC*, 2019, pp. 913–928.
- [23] J. Handy and T. Coughlin, "Optane's dead: Now what?" *Computer*, vol. 56, no. 3, pp. 125–130, Mar. 2023.
- [24] J. Henkel et al., "Special session—non-volatile memories: Challenges and opportunities for embedded system architectures with focus on machine learning applications," in *Proc. Int. Conf. Compil., Archit., Synth. Embedded Syst.*, Sep. 2023, pp. 11–20.
- [25] Y. Hua, X. Liu, and I. Stoica, "The golden rule of big memory: Persistence is not harmful," *Commun. ACM*, vol. 69, no. 5, pp. 51–55, Apr. 2026, doi: 10.1145/3769003.
- [26] C.-H. Huang and I. G. Thakkar, "Mitigating write disturbance in phase change memory architectures: Work-in-progress," in *Proc. Int. Conf. Compilers, Archit. Synth. Embedded Syst. Companion*, Oct. 2019, pp. 1–2.
- [27] L. Kernel. *Direct Access for Files*. Accessed: Aug. 29, 2026. [Online]. Available: <https://www.kernel.org/doc/Documentation/filesystems/dax.txt>
- [28] P.-Å. Larson and M. Krishnan, "Memory allocation for long-running server applications," *ACM SIGPLAN Notices*, vol. 34, no. 3, pp. 176–185, Mar. 1999.
- [29] S. Lee, T. Johnson, and E. Raman, "Feedback directed optimization of TCMalloc," in *Proc. Workshop Memory Syst. Perform. Correctness*, Jun. 2014, pp. 1–8.
- [30] R. Leite and R. Rocha, "A modern and competitive lock-free dynamic memory allocator," in *Proc. INForum*, 2018, pp. 230–243.
- [31] D. Li, B. Reidys, J. Sun, T. Shull, J. Torrellas, and J. Huang, "UniHeap: Managing persistent objects across managed runtimes for non-volatile memory," in *Proc. 14th ACM Int. Conf. Syst. Storage*, Jun. 2021, pp. 1–12.
- [32] Z. Mao, S. Zheng, L. Huang, and Y. Shen, "A DAX-enabled mmap mechanism for log-structured in-memory file systems," in *Proc. IEEE 36th Int. Perform. Comput. Commun. Conf. (IPCCC)*, Dec. 2017, pp. 1–8.
- [33] I. MicroQuill. (2014). *Shbench*. [Online]. Available: <https://www.microquill.com/smartheap/shbench>
- [34] Y. Mo, J. Jia, P. Li, and Y. Hua, "An efficient labeled memory system for learned indexes," *Fundam. Res.*, vol. 4, no. 3, pp. 651–659, May 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2667325822002230>
- [35] I. Oukid, D. Booss, A. Lespinasse, W. Lehner, T. Willhalm, and G. Gomes, "Memory management techniques for large-scale persistent-main-memory systems," *Proc. VLDB Endowment*, vol. 10, no. 11, pp. 1166–1177, Aug. 2017.
- [36] I. Oukid, J. Lasperas, A. Nica, T. Willhalm, and W. Lehner, "FPTree: A hybrid SCM-DRAM persistent and concurrent B-tree for storage class memory," in *Proc. Int. Conf. Manage. Data*, Jun. 2016, pp. 371–386.
- [37] A. Papagiannis, G. Xanthakis, G. Saloustros, M. Marazakis, and A. Bilas, "Optimizing memory-mapped I/O for fast storage devices," in *Proc. ATC*, 2020, pp. 813–827.
- [38] P. Ramalhete, A. Correia, and P. Felber, "Efficient algorithms for persistent transactional memory," in *Proc. 26th ACM SIGPLAN Symp. Princ. Pract. Parallel Program.*, Feb. 2021, pp. 1–15.

- [39] A. Raybuck, T. Stamler, W. Zhang, M. Erez, and S. Peter, "HeMem: Scalable tiered memory management for big data applications and real NVM," in *Proc. ACM SIGOPS 28th Symp. Operating Syst. Princ.*, Oct. 2021, pp. 392–407.
- [40] A. Reaugh and S. A. Salehi, "Work-in-progress: Efficient low-latency near-memory addition," in *Proc. Int. Conf. Compil., Archit., Synth. Embedded Syst. (CASES)*, Oct. 2022, pp. 33–34.
- [41] N. Riekenbrauck, M. Weisgut, D. Lindner, and T. Rabl, "A three-tier buffer manager integrating CXL device memory for database systems," in *Proc. IEEE 40th Int. Conf. Data Eng. Workshops (ICDEW)*, May 2024, pp. 395–401.
- [42] S. Scargall, *Programming Persistent Memory: A Comprehensive Guide for Developers*. Cham, Switzerland: Springer, 2020.
- [43] S. Schneider, C. D. Antonopoulos, and D. S. Nikolopoulos, "Scalable locality-conscious multithreaded memory allocation," in *Proc. 5th Int. Symp. Memory Manage.*, Jun. 2006, pp. 84–94.
- [44] D. Schwalb, T. Berning, M. Faust, M. Dreseler, and H. Plattner, "nvm_malloc: Memory Allocation for NVRAM," in *Proc. Adms@VLDB*, 2015, pp. 61–72.
- [45] H. Shacham, M. Page, B. Pfaff, E.-J. Goh, N. Modadugu, and D. Boneh, "On the effectiveness of address-space randomization," in *Proc. 11th ACM Conf. Comput. Commun. Secur.*, Oct. 2004, pp. 298–307.
- [46] L. Torvalds, *Everything is a File*. Accessed: Aug. 29, 2026. [Online]. Available: https://yarchive.net/comp/linux/everything_is_file.html
- [47] H. Volos, A. J. Tack, and M. M. Swift, "Mnemosyne: Lightweight persistent memory," in *Proc. ASPLOS*, 2011, pp. 91–104.
- [48] P. R. Wilson, M. S. Johnstone, M. J. Neely, and D. B. Boles, "Dynamic storage allocation: A survey and critical review," in *Proc. IWMM*, 1995, pp. 1–116.
- [49] X. Xiang, Y. Hua, and H. Xu, "PMA: A persistent memory allocator with high efficiency and crash consistency guarantee," in *Proc. IEEE 41st Int. Conf. Comput. Design (ICCD)*, Nov. 2023, pp. 182–189.
- [50] J. Xu et al., "NOVA-fortis: A fault-tolerant non-volatile main memory file system," in *Proc. 26th Symp. Operating Syst. Princ.*, Oct. 2017, pp. 478–496.
- [51] W. Yan and X. Zhang, "Improving the relationship between B+-tree and memory allocator for persistent memory," in *Proc. IEEE 40th Int. Conf. Data Eng. (ICDE)*, May 2024, pp. 3906–3919.
- [52] J. Yang, B. Li, and D. J. Lilja, "Exploring performance characteristics of the optane 3D xpoint storage technology," *ACM Trans. Model. Perform. Eval. Comput. Syst.*, vol. 5, no. 1, pp. 1–28, Mar. 2020.
- [53] J. Yang, B. Li, J. Yuan, Z. Shen, D. Du, and D. Lilja, "Work-in-progress: ExpCache: Online-learning based cache replacement policy for non-volatile memory," in *Proc. Int. Conf. Compil., Archit., Synth. Embedded Syst. (CASES)*, Oct. 2022, pp. 7–8.
- [54] J. Zeng, J. Jeong, and C. Jung, "Persistent processor architecture," in *Proc. 56th Annu. IEEE/ACM Int. Symp. Microarchitecture*, Oct. 2023, pp. 1075–1091.
- [55] J. Zeng, T. Zhang, and C. Jung, "Compiler-directed whole-system persistence," in *Proc. ACM/IEEE 51st Annu. Int. Symp. Comput. Archit. (ISCA)*, Jun. 2024, pp. 961–977.
- [56] B. Zhang, S. Zheng, Z. Qi, and L. Huang, "NBTree: A lock-free PM-friendly persistent B+-tree for eADR-enabled PM systems," *Proc. VLDB Endowment*, vol. 15, no. 6, pp. 1187–1200, Feb. 2022.
- [57] W. Zhang, X. Zhao, S. Jiang, and H. Jiang, "ChameleonDB: A key-value store for optane persistent memory," in *Proc. 16th Eur. Conf. Comput. Syst.*, Apr. 2021, pp. 194–209.
- [58] Y. Zhong et al., "Managing memory tiers with CXL in virtualized environments," in *Proc. OSDI*, 2024, pp. 37–56.
- [59] Y. Zhou, J. Zeng, and C. Jung, "LightWSP: Whole-system persistence on the cheap," in *Proc. 57th IEEE/ACM Int. Symp. Microarchitecture (MICRO)*, Nov. 2024, pp. 215–230.
- [60] P. Zuo, Y. Hua, and J. Wu, "Write-optimized and high-performance hashing index scheme for persistent memory," in *Proc. OSDI*, 2018, pp. 461–476.



Yujie Hu (Graduate Student Member, IEEE) received the B.E. degree in electronic information from the Huazhong University of Science and Technology, Wuhan, China, in 2023, where she is currently pursuing the Ph.D. degree in computer science.

Her current research interests include nonvolatile memory systems, CXL memory, and disaggregated memory.



Yu Hua (Senior Member, IEEE) is a Professor with the Huazhong University of Science and Technology, Wuhan, China. His research interests include cloud storage systems, file systems, nonvolatile memory architectures, and big memory.



Xiangyu Xiang received the B.E. and M.S. degrees in computer science from the Huazhong University of Science and Technology, Wuhan, China, in 2022 and 2025, respectively.

His research interests include nonvolatile memory systems.



Aoyang Tong received the B.E. degree in computer science from the Huazhong University of Science and Technology, Wuhan, China, in 2023, where he is currently pursuing the Ph.D. degree in computer science.

His research interests include SmartNICs, learned indexes, and nonvolatile memory systems.



Xue Liu (Fellow, IEEE) received the Ph.D. degree in computer science from the University of Illinois at Urbana-Champaign, Champaign, IL, USA, in 2006.

He is currently a Professor with the School of Computer Science, McGill University, Montreal, QC, Canada. His research interests include intelligent computing and communications systems, sustainable computing, and smart energy systems, AI, machine learning, and applications, and recommender systems.